Analysis of detection algorithms for SIRENA

Check results of possible input parameters

- samplesUp (fixed): number of consecutive samples in the derivative above the threshold
- samplesDown (fixed): number of consecutive samples in the derivative below the threshold to start triggering again
- threshold (fixed): value to be crossed by the derivative
- window: size (samples) of the window to calculate average derivative and do a subtraction
  Ex. window = 3 :

  ```
  deriv[i] => deriv[i] - mean(deriv[i-1], deriv[i-2], deriv[i-3])
  ```
- offset: offset (samples) of the subtracting window
  Ex. window = 3 && offset = 2 :

  ```
  deriv[i] => deriv[i] - mean(deriv[i-3], deriv[i-4], deriv[i-5])
  ```

Procedure

1. (*external*) XIFUSIM files with 100 pairs of pulses are simulated:

   ```
   Eprimary = [0.2, 0.5, 1, 2, 3, 4, 5, 6, 7, 8, 9 , 10, 11 ,12]
   Esecondary = [0.2, 0.5, 1, 2, 3, 4, 5, 6, 7, 8, 9 , 10, 11 ,12]
   Separations =
[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,25,30,35,40,45,50,60,70,80,90,100,110,126]
   ```

2. (*external*) SIRENA reconstructed (xifusim) files using combinations of window and offset:

   ```
   samplesUp = 3
   samplesDown = 2
   threshold = 6
   window = [0, 1, 2, 3, 4, 5, 6, 10, 15, 20]
   offset = [0, 1, 2, 3, 4, 5, 6]
   The combination window=0 & offset=0 corresponds to the traditional method (no derivative
subtraction)
   ```

3. (*external*) For each window/offset combination a pickle object (file) is created with the following information:

   ```
   | separation | energy1 | energy2 | window | offset | ndetected | nfake |
   ```

4. Analysis in this notebook:

   a) (Optional): calculate **weights** for separations according to (Poiss) mathematical prob distribution
   b) (Optional): calculate **probability distribution** of energies in the pairs according to a source spectral model
   c) (Optional): create a source probability cube to correct the simulations cube that has a uniform distribution of energies and separations
   d) For each window/offset:

   - Read pickle file
   - (Optionally) create a FITS data cube of number of detected photons: AXIS1-ENERGY1, AXIS2-ENERGY2, AXIS3-separations
   - (Optionally) multiply the simulated cube by the probability cube to work with realistic simulations
   - Save the fraction of detections: numpy[separations, window, offset] (**Warning**: indexing in numpy and FITS is reversed)
   - Plot an image of E2 vs E1 for a given separation (data cube slice)

   e) Create a mosaic of images with all the windows and offsets
   f) Write a FITS cube with fraction of detections: AXIS1-window, AXIS2-offset, AXIS3-separations
   d) Collapse the cube in energies (sum of all fractions) and separations (mean value): plot image of fraction of detections (offset vs window)

Pseudo-code (in case of source weighting):

1. Build temporal-separation weights

Input:

```
separations Δt_k
temporal rates λ_p for the top pixels

For each pixel p:
    For each separation bin k:
        Compute bin probability:
prob_sep[ipix,isep]    Prob_p(k) = exp(-λ_p * s1_k) - exp(-λ_p * s2_k)
    Normalize:
weights_sep[ipix,isep]    w_sep_p(k) = Prob_p(k) / mean_k[Prob_p(k)]
```

Check pixel-to-pixel consistency of w_sep_p(k):

```
If all weight curves nearly identical => Select main pixel p* for further use
```

Output:

w_sep(k) = w_sep_p*(k)

2. Build spectral probability map

Input:

```
Photon energies E of the most impacted pixel, aggregated across simulations
```

Construct 1D energy histogram: ener_hist = np.histogram(E1E2_energies)

```
H(E_i) = count of photons in bin i
P(E_i) = H(E_i) / sum_i H(E_i)
```

in code normalization is done in the 2D histogram

Construct 2D spectral probability:

```
W(E1_i, E2_j) = P(E1_i) * P(E2_j)
Normalize W so that sum_{i,j} W = 1
```

ener_hist2d = np.outer(ener_hist,ener_hist)

prob_matrix = ener_histo2d/ sum(ener_histo2d)

Output:

W(E1,E2)

3. Construct the joint probability cube

For each separation k:

```
For each energy bin pair (i,j):
    prob_cube(k,i,j) = w_sep(k) * W(E1_i, E2_j)
```

weights_sep[0, isep] * prob_matrix

Result: prob_cube is a 3D array over (Δt_k, E1_i, E2_j)

4. Build SIRENA detection cubes for each (window, offset)

For each SIRENA configuration (w,o):

```
Read detection events from simulation output:
    For each detection:
        extract separation k, energies (i,j), ndetected

Construct detection cube:
    pairs_cube(k,i,j) = sum of ndetected for all events in that bin

Mark separation bins not present in the simulation:
    pairs_cube(k,:,:)=NaN
```

5. Apply the temporal × spectral probability cube

For each (w,o):

```
weighted_cube(k,i,j) = pairs_cube(k,i,j) * prob_cube(k,i,j)

Normalize per separation:
    fraction(k,i,j) = weighted_cube(k,i,j) / Nsim_per_bin
```

Key point:
fraction(k,i,j) is a fraction of detected pulses in that bin, not a probability density.

6. Collapse the weighted cube over the energy plane

For each (w,o) and each separation k:

```
slice_fraction(k) = sum_{i,j} fraction(k,i,j)
```

Each slice_fraction(k) is already weighted by w_sep(k) and W(E1,E2).

- Each slice_fraction(k) is already normalized by the probability cube.
- The mean combines these weighted fractions while ignoring missing separation bins (NaNs).
- A sum would double-count probability mass and yield values > 1, which contradicts the definition of a fraction of detected pulses.

7. Collapse over separations

For each (w,o):

```
final_value(o,w) = nanmean_k [ slice_fraction(k) ]
```

> **NOTE**:
> to convert this notebook into a Python script (for Slurm), just "*Export as*" -> Python and comment the line:
> `%matplotlib widget`

Import modules

In [ ]:
```
# import python modules
import argparse
import os
import glob
import matplotlib.pyplot as plt
from matplotlib.colors import SymLogNorm, LinearSegmentedColormap
import numpy as np
# import for pickling
import pickle
from astropy import table
from astropy.io import fits
import auxiliary as aux
import matplotlib.colors as mcolors
import mplcursors
import ipywidgets as widgets
%matplotlib widget
#%matplotlib qt
#%config InlineBackend.figure_format = 'retina'
```

Running Jupyter or Python script?

- It tries to call get_ipython() (only available in IPython environments, like Jupyter).
- If the shell class name is "ZMQInteractiveShell", it confirms that you're in a Jupyter notebook (or JupyterLab).
- If it's a regular Python interpreter, the function returns False.

In [ ]:
```
# parameter handling
def get_parameters():
```

```python
    """
    Get parameters for pairs detection analysis.
    If running in a Jupyter Notebook, use default parameters.
    If running as a script (e.g., SLURM), parse command line arguments.
    """
    global th, sUp, sDown, windows, offsets, relevant_separations, xifu_config, create_cubes, sep_for_plot_mosaic

    # Check if running in a Jupyter Notebook or as a script
    if aux.is_notebook():
        # Default parameters for interactive use
        print("Running in notebook mode for pairs detection analysis")
        return {
            "threshold": 6.0, # threshold for detection
            "samplesUp": 3,  # samples up for detection
            "samplesDown": 2, # samples down for detection
            "windows": [0,1, 2, 3, 4, 5, 6, 10, 15, 20], # subtraction derivative window for detection
            #"windows": [1, 2, 3, 4, 5, 6], # subtraction derivative window for detection
            "offsets": [0,1, 2, 3, 4, 5, 6],  # offset for subtraction window
            #"offsets": [1, 2, 3, 4, 5, 6],  # offset for subtraction window
            "relevant_separations": [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,25,30,35,40,45,50,60,70,80
            "config_version": 'v5_20250621',  # XIFU configuration
            "sep_for_plot_mosaic": 8, # samples separation for plotting the mosaic of slices of the data cube (if
            "win_collapsed_cube": [0,1,2,3,4,5,6], # window sizes for the collapsed cube
            "use_sim_source": True,  # use simulated source for cube correction
            "sim_source_flux": 0.5,  # flux of a simulated source (to get the "real" distribution of pairs
            "verbose": 1  # verbosity flag
        }
    else:
        # Parameters from command line (e.g., for SLURM)
        parser = argparse.ArgumentParser(
            description='Execute the python script for pairs detection analysis',
            prog='execute_create_cubes.py')
        parser.add_argument('--windows', required=False, type=int,
                            nargs='*', default=[0, 1, 2, 3, 4, 5, 10, 15, 20],
                            help='Subtraction derivative window for detection')
        parser.add_argument('--offsets', required=False, type=int,
                            nargs='*', default=[0, 1, 2, 3, 4, 5],
                            help='Offset for subtraction window')
        parser.add_argument('--threshold', required=False, type=float, default=0.5,
                            help='Threshold for detection')
        parser.add_argument('--samplesUp', required=False, type=int, default=2,
                            help='Samples up for detection')
        parser.add_argument('--samplesDown', required=False, type=int, default=2,
                            help='Samples down for detection')
        parser.add_argument('--config_version', required=False, type=str, default='v5_20250621',
                            help='XIFU configuration version')
        parser.add_argument('--relevant_separations', required=False, type=int,
                            nargs='*', default=[8, 20, 50, 126, 317, 797],
                            help='Relevant separations for the analysis')
        parser.add_argument('--sep_for_plot_mosaic', required=False, type=int, default=-1,
                            help='Samples separation for plotting the mosaic of slices of the data cube (if negati
        parser.add_argument('--win_collapsed_cube', required=False, type=int,
                            nargs='*', default=[0, 1, 2, 3, 4, 5, 6],
                            help='Window sizes for the collapsed cube')
        parser.add_argument('--sim_source_flux', required=False, type=float, default=0.,
                            help='Flux of a simulated source (to get the "real" distribution of pairs)')
        parser.add_argument('--use_sim_source', action='store_true',
                            help='Use simulated source for cube correction')
        parser.add_argument('--verbose', required=False, type=int, default=0,
                            help='Verbosity flag (0: no output, 1: some output, 2: detailed output)')

        args = parser.parse_args()

        return vars(args)
```

Get parameters

In [ ]:
```python
params = get_parameters()
th = params['threshold']
sUp = params['samplesUp']
sDown = params['samplesDown']
windows = params['windows']
offsets = params['offsets']
xifu_config = params['config_version']
relevant_separations = params['relevant_separations']
```

```
sep_for_plot_mosaic = params['sep_for_plot_mosaic']
windows_for_collapsed_cube = params['win_collapsed_cube']
sim_source_flux = params['sim_source_flux']
use_sim_source = params['use_sim_source']
aux.verbose = params['verbose']
print(f"Parameters for pairs detection analysis: {params}")
```

Secondary parameters

In [ ]:
```
sampling_rate = 130210 # Hz
min_detected = 100 # minimum number of detected pulses (there are 100 pairs simulations; first pulse will always b
max_detected = 200 # maximum number of detected pulses (there are 100 pairs simulations; so max is 200)
energy_bin_centers = [0.2, 0.5, 1., 2., 3., 4., 5., 6., 7., 8., 9., 10., 11., 12.]
energy_bin_edges = [energy_bin_centers[0] - (energy_bin_centers[1] - energy_bin_centers[0]) / 2] + \
             [(energy_bin_centers[i] + energy_bin_centers[i+1]) / 2 for i in range(len(energy_bin_centers) - 1)] +
             [energy_bin_centers[-1] + (energy_bin_centers[-1] - energy_bin_centers[-2]) / 2]
seps_bin_centers = relevant_separations
seps_bin_edges = [seps_bin_centers[0] - (seps_bin_centers[1] - seps_bin_centers[0]) / 2] + \
             [(seps_bin_centers[i] + seps_bin_centers[i+1]) / 2 for i in range(len(seps_bin_centers) - 1)] + \
             [seps_bin_centers[-1] + (seps_bin_centers[-1] - seps_bin_centers[-2]) / 2]
print(f"Separations bin edges: {seps_bin_edges}")

EURECA_dir = "/dataj6/ceballos/INSTRUMEN/EURECA/"
analysis_dir = f"{EURECA_dir}/TN350_detection/2024_revision/"
pickles_dir = f"{EURECA_dir}/ERESOL/CEASaclay/July2025_v5_v20250621_offsetWindow/"
old_separations = [8, 20, 50, 126]  # default meaningful separations for window=0,10,15,20; if different sep is pl
old_windows = [0, 10, 15, 20]  # old windows for the analysis (previous to fine grid simulations between 1 and 6)
nsimulated_1e1_1e2_1sep = 200 # number of simulated events for each separation an a given combination of E1 and E2
nsimulated_1sep = len(energy_bin_centers) * len(energy_bin_centers) * nsimulated_1e1_1e2_1sep  # total number of s
nsims_sixte = 100  # number of SIXTE simulations to consider
ntotal_pixels = 1504 # total number of XIFU pixels
ntop_pixels = 10  # number of top pixels to consider for the analysis
```

In [ ]:
```
def on_hover(event, text_obj, data, bin_edges, ax, fig):
    if event.inaxes != ax:
        text_obj.set_text('')
        fig.canvas.draw_idle()
        return

    if event.xdata is None or event.ydata is None:
        text_obj.set_text('')
        fig.canvas.draw_idle()
        return

    x, y = event.xdata, event.ydata

    # Find the bin index for x and y using np.searchsorted
    col = np.searchsorted(bin_edges, x) - 1
    row = np.searchsorted(bin_edges, y) - 1

    # Check bounds
    if 0 <= row < data.shape[0] and 0 <= col < data.shape[1]:
        val = data[row, col]
        x_center = bin_edges[col]
        y_center = bin_edges[row]
        text_obj.set_text(f'E1: {x_center:.1f} keV\nE2: {y_center:.1f} keV\nProb: {val:.2e}')
    else:
        text_obj.set_text('Outside bounds')

    fig.canvas.draw_idle()
```

Get time distribution weights (using Poisson stats)

This will be used to give different weights to the pairs separations in the test data cube
We will check that these weights are fully similar for all the pixels (at the end we may want to calculate only the weights for the most populated pixels and all the simulations)

- Get the list of the most populated pixels (Use only simulation 1)
- calculate separation probability (& weights): $prob = exp(-\lambda * s1) - exp(-\lambda * s2)$
- plot weights for different countrates in sim_1
- as they are similar, use only the count rate of the pixel with more counts for the rest of the analysis

In [ ]:
```
if use_sim_source:
    sim_source_dir = f"{analysis_dir}/{xifu_config}/flux{sim_source_flux:.2f}mcrab"
    top_counts_file = f"{sim_source_dir}/top_counts.pkl"
    # get list of impacted pixels
    sim1_piximpact_files = glob.glob(f"{sim_source_dir}/sim_1/crab_flux*_Emin2_Emax10_exp*_RA0.0_Dec0.0_*_*pixel*_
    if len(sim1_piximpact_files) == 0:
        raise ValueError(f"No pixel impact files found in {sim_source_dir}/sim_{isim}/ for simulation {isim}.")
    # sort files by size (descending order) and get the top most large files
    piximpact_files = sorted(sim1_piximpact_files, key=os.path.getsize, reverse=True)[:ntop_pixels]
    # get exposure time from the filename
    exposure_time = float(piximpact_files[0].split('_exp')[1].split('_')[0])
    # get pixel numbers from filenames
    top_pixel_nums = [int(f.split('_')[-2].replace('pixel', '')) for f in piximpact_files]

    if os.path.exists(top_counts_file):
        aux.vprint(f"Loading top counts from {top_counts_file}")
        with open(top_counts_file, 'rb') as f:
            top_counts = pickle.load(f)
    else:
        print(f"Top counts file {top_counts_file} does not exist. Will create it.")
        # Use "real" source simulations to get the impacting photons

        #initialize to 0 1D numpy array for counts in most populated pixels
        counts_pixel = np.zeros(ntotal_pixels)

        for i, piximpact_file in enumerate(piximpact_files):
            pixel_num = top_pixel_nums[i]
            piximpact_file = piximpact_files[i]
            print(f"Reading impact file #{i+1}/{len(piximpact_files)}: {piximpact_file}")
            # read number of events in the pixel from keyword NAXIS2
            with fits.open(piximpact_file) as hdul:
                counts_pixel[pixel_num-1] = hdul[1].header['NAXIS2']

        print("\nTop populated pixels (by number of counts):")
        for pixel_num in top_pixel_nums:
            ipixel_num = pixel_num - 1
            aux.vprint(f"Counts for pixel {pixel_num}: {counts_pixel[ipixel_num]}")


        # initialize to 0 2D numpy array for counts in most populated pixels
        top_counts = np.zeros((ntop_pixels, nsims_sixte))
        # get the mean count rate (using all simulations) for the top most populated pixels
        top_ctrate_sam = np.zeros(ntop_pixels)  # mean count rate per sample

        for ipix in range(len(top_pixel_nums)):
            pixel_num = top_pixel_nums[ipix]
            aux.vprint(f"Processing pixel {pixel_num}")
            # initialize counts for the impacted pixels
            for i in range(nsims_sixte):
                isim = i + 1
                piximpact_file = glob.glob(f"{sim_source_dir}/sim_{isim}/crab_flux*_Emin2_Emax10_exp*_RA0.0_Dec0.0
                if len(piximpact_file) > 1:
                    raise ValueError(f"Multiple pixel impact files found for pixel {pixel_num} in simulation {isim
                elif len(piximpact_file) == 1:
                    piximpact_file = piximpact_file[0]
                    # read number of events in the pixel from keyword NAXIS2
                    with fits.open(piximpact_file) as hdul:
                        top_counts[ipix, i] = hdul[1].header['NAXIS2']

        # save the top counts in a pickle file for future use
        with open(top_counts_file, 'wb') as f:
            pickle.dump(top_counts, f)
    # calculate the mean count rate (cts per sample) for each pixel
    top_ctrate_sam = np.mean(top_counts, axis=1) / (exposure_time * sampling_rate)
```

In [ ]:
```
if use_sim_source:
    # calculate the probability of each separation for the top most populated pixels
    # Probability of each separation (see Nico's notebook)
    #            prob = exp(-lambda*s1) - exp(-lambda*s2)
    #            lambda = count_rate / sampling_rate
    #            s1: left border of the separation bin -> sep - 0.5
    #            s2: right border of the separation bin -> sep + 0.5
    prob_sep = np.zeros((ntop_pixels, len(relevant_separations)))
    weights_sep = np.zeros((ntop_pixels, len(relevant_separations)))
    for ipix in range(ntop_pixels):
        # calculate the probability of each separation for the pixel
```

```
        for isep, sep in enumerate(relevant_separations):
            prob_sep[ipix, isep] = np.exp(-top_ctrate_sam[ipix] * (sep-0.5)) - np.exp(-top_ctrate_sam[ipix] * (sep
        # get the weights for the separations (for each pixel)
        weights_sep[ipix, :] = prob_sep[ipix, :] / np.mean(prob_sep[ipix, :])
```

In [ ]:
```
# plot the probability of each separation for the 10 top most populated pixels
if use_sim_source:
    fig, ax = plt.subplots(figsize=(10, 6))
    for ipix in range(ntop_pixels):
        pixel_num = top_pixel_nums[ipix]
        counts_in_pixel = top_counts[ipix, :].sum()
        ax.plot(relevant_separations, weights_sep[ipix, :], label=f"Pixel {pixel_num} ({counts_in_pixel/1000.:.1f}
    ax.set_xlabel("Separation (samples)")
    ax.set_ylabel("Weight")
    ax.set_title(f"Weights of each separation for the {ntop_pixels} top most populated pixels")
    ax.legend()
    plt.show()
```

Get energy distribution probability (using the simulated source spectrum)

Once we have seen that the probability curves (weights of different separations) are essentially identical for all the main pixels (i.e. for small separations,
where problems can arise, it does not change a lot with the count rate), we'll use only the most impacted pixel in the SIXTE simulations to proceed with the analysis.

Use the list of energies of the photons in the pixel to create an energy distribution map -> probability distribution of energies

We will use this probability map to correct the number of pairs simulated in the test cube according to the input spectrum

Get distribution of energies

In [ ]:
```
if use_sim_source:
    # take the first pixel as the main pixel (more counts)
    main_pixel_num = top_pixel_nums[0]  # first pixel in the list of top pixels
    main_pixel_energies_file = f"{sim_source_dir}/pixel{main_pixel_num}_energies.pkl"
    if os.path.exists(main_pixel_energies_file):
        aux.vprint(f"Loading main pixel energies from {main_pixel_energies_file}")
        with open(main_pixel_energies_file, 'rb') as f:
            E1E2_energies = pickle.load(f)
    else:
        # take the last pixel as the main pixel (more counts)
        vprint("Main pixel energies file does not exist. Will create it.")
        main_pixel_num = top_pixel_nums[0]  # first pixel in the list of top pixels
        #index_main_pixel = main_pixel_num - 1  # index of the main pixel in the counts array
        main_pixel_counts = top_counts[0, :].sum()  # total counts in the main pixel
        vprint(f"Using main pixel for analysis: {main_pixel_num} with {main_pixel_counts} counts")

        all_energies = []  # to store energies of all simulations

        # Get a "real" spectrum of the simulated source in one (maximum) pixel: read the energies of the photons i
        vprint(f"Simulated source flux: {sim_source_flux} mCrab")
        for isim in range(1,nsims_sixte+1):
            piximpact_file = glob.glob(f"{sim_source_dir}/sim_{isim}/crab_flux*_*_pixel{main_pixel_num}_piximpact.
            if len(piximpact_file) > 1:
                raise ValueError(f"Multiple piximpact files for pixel {main_pixel_num} found for simulated source:
            elif len(piximpact_file) == 0:
                raise FileNotFoundError(f"No piximpact file found for simulated source in pixel {main_pixel_num} a
            piximpact_file = piximpact_file[0]  # take the file found
            #print(f"Reading piximpact file for simulated source: {piximpact_file}")

            with fits.open(piximpact_file) as hdul:
                aux.vprint(f"Reading piximpact file for simulation {isim} \r", end="")
                # Assuming the first HDU contains the data
                data = hdul[0].data
                # read ENERGY of photons
                energies = hdul[1].data['ENERGY']
                all_energies.append(energies)
        # concatenate all energies from different simulations
        E1E2_energies = np.concatenate(all_energies)  # concatenate all energies from different simulations
        if len(E1E2_energies) != main_pixel_counts:
            raise ValueError(f"Inconsistent number of energies found in the simulated source for pixel {main_pixel_
        # save the energies of the photons in the main pixel
        vprint(f"Saving main pixel energies to {main_pixel_energies_file}")
```

```
        with open(main_pixel_energies_file, 'wb') as f:
            pickle.dump(E1E2_energies, f)
```

Plot energy distribution map (probability)

In [ ]:
```
if use_sim_source:
    # Compute 1D histogram of energies
    ener_hist, _ = np.histogram(E1E2_energies, bins=energy_bin_edges)

    # Now create the 2D outer product to form the 2D histogram image
    ener_hist2d = np.outer(ener_hist, ener_hist)

    # Normalize to make it a probability image
    prob_matrix = ener_hist2d / np.sum(ener_hist2d)
    # check that the probability matrix is normalized
    if np.sum(prob_matrix) != 1:
        raise ValueError("The probability matrix is not normalized. Check the energies of the photons in the main
```

In [ ]:
```
if use_sim_source:
    # Plot the image
    fig, ax = plt.subplots(figsize=(8, 6))
    # Set the colormap and normalization
    vmin = np.min(prob_matrix[prob_matrix > 0])  # minimum value for normalization
    vmax = np.max(prob_matrix)  # maximum value for normalization
    norm = mcolors.LogNorm(vmin=vmin, vmax=vmax)
    # Create the image with the specified colormap and normalization
    X, Y = np.meshgrid(energy_bin_edges, energy_bin_edges)
    # Create the image with the specified colormap and normalization
    source_mesh = ax.pcolormesh(X, Y, prob_matrix, cmap='viridis', norm=norm, shading='auto')
    ax.set_xlabel('Energy1')
    ax.set_ylabel('Energy2')
    ax.set_title(f'Energies probability distribution (Crab model, flux={sim_source_flux:.2f} mCrab)')
    fig.colorbar(source_mesh, ax=ax, label='Probability')

   # Create a text object for displaying values
    text_obj = ax.text(0.02, 0.98, '', transform=ax.transAxes,
                       bbox=dict(boxstyle='round', facecolor='white', alpha=0.8),
                       verticalalignment='top')
    # Connect the hover event
    cid = fig.canvas.mpl_connect('motion_notify_event', lambda event: on_hover(event, text_obj, prob_matrix, energ

    plt.show()
```

Create the probability cube (energy*separation)

In [ ]:
```
if use_sim_source:
    # multiply the prob_matrix by the prob_sep and create a cube
    prob_cube = np.zeros((len(seps_bin_edges)-1, len(energy_bin_edges)-1, len(energy_bin_edges)-1))
    for isep, sep in enumerate(relevant_separations):
        #prob_cube[isep:, :, :] = prob_matrix * weights_sep[0, isep]  # use the top pixel probabilities
        prob_cube[isep, :, :] = prob_matrix * weights_sep[0, isep]  # use the top pixel temporal weight for this s
    # create a FITS file with the probability cube
    prob_cube_file = f"{sim_source_dir}/probability_cube_pixel{main_pixel_num}.fits"
    hdu = fits.PrimaryHDU(prob_cube)
    hdu.header['XIFU_CFG'] = xifu_config
    hdu.header['FLUX'] = sim_source_flux
    hdu.header['MAXPIXEL'] = main_pixel_num
    hdu.writeto(prob_cube_file, overwrite=True)
    aux.vprint(f"Probability cube shape (seps,ener1, ener2): {prob_cube.shape} saved to {prob_cube_file}")
```

In [ ]:
```
print(np.min(prob_matrix), np.max(prob_matrix), np.sum(prob_matrix))
print(np.min(prob_cube), np.max(prob_cube), np.sum(prob_cube))
```

Analyse results for window/offset variations

1. SIRENA reconstruction results (of uniform pairs) are saved in pickle files for each combination of window and offset
2. Create Data cubes (optionally save them to FITS files):

| NAXIS3(sep)
|
|____ NAXIS1(e1)

/
NAXIS2(e2)

3. Save fractions of detected pulses [separation, window, offset]
4. Plot E2 vs E1 mosaic of images for a given separation

In [ ]:
```
# initialize some parameters for the analysis
nrows_mosaic = len(offsets)  # number of rows for the mosaic plot
ncols_mosaic = len(windows)  # number of columns for the mosaic plot

print(f"Mosaic plot will have {nrows_mosaic} rows and {ncols_mosaic} columns based on the provided offsets and win

# initialize a numpy array 3-D (sep, offset, window) for fraction of detected pulses (opt. sim source corrected) w
cube_e1e2collapsed = np.nan * np.full((len(relevant_separations), len(offsets), len(windows_for_collapsed_cube)),
```

Create the CUBES (pairs, corrected, collapsed) for all win/off combinations

- Avoid impossible combinations of win/off (not simulated or not meaningful)
- for each win/off read pickel file with number of detections
- create (read) cube of detections of pairs as an histogramdd (sep, e1, e2)
- replace with NaN those sep slices that have not been simulated for the current win/off
- if using simulated source, correct pairs cube with (e1e2 matrix probability) * (poiss weights for separations) -> save cube
- create collapsed cube (summing up in e1,e2 axes) to plot final map of win/off combinations

In [ ]:
```
use_separations = relevant_separations  # use the relevant separations for the analysis
for io in range(len(offsets)):
    for iw in range(len(windows)):
        off = offsets[io]
        win = windows[iw]
        if win == 0 and off > 0:
            # skip this combination, as it is not meaningful
            aux.vprint(f"    Skipping window {win} with offset {off} as it is not applicable.")
            continue
        if win in old_windows and off > 5:
            aux.vprint(f"    Skipping window {win} with offset {off} as it has not been simulated.")
            continue
        aux.vprint(f"Offset: {off}, Window: {win}")
        # check if cube exists
        pairs_cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/pairs_cubes/pairs_cube_detections_win{w
        if not os.path.exists(pairs_cube_detections_iw_io_file):
            aux.vprint(f"    Pairs cube detections file {pairs_cube_detections_iw_io_file} does not exist. Will cr

            pickle_file = f'{pickles_dir}/detectedFakes_win{win}_off{off}.pkl'
            # read detection data from the pickle file
            with open(pickle_file, 'rb') as f:
                data = pickle.load(f)
                #print(data)
                if win == 0 and off == 0:
                    # for window=0 and offset=0, we have a different data structure
                    data_table = table.Table(rows=data, names=('separation', 'energy1', 'energy2', 'samplesDown',
                    data_filtered = data_table[(data_table['threshold'] == th) & (data_table['samplesUp'] == sUp)
                else:
                    data_table = table.Table(rows=data, names=('separation', 'energy1', 'energy2', 'window', 'offs
                    data_filtered = data_table.copy()

            # get only separations that are in the relevant separations list
            data_filtered = data_filtered[np.isin(data_filtered['separation'], use_separations)]
            #print(f"Filtered data: {data_filtered}")

            #look for rows where nfake > 0
            data_filtered_nfake = data_filtered[data_filtered['nfake'] > 0]
            if len(data_filtered_nfake) > 0:
                aux.vprint(f"Filtered data with nfake > 0: {data_filtered_nfake}")

            # create a data cube for each pickle file (and optionally save it to a FITS file)
            #  (use histogramdd to create a cube of detections)
            e1_for_pairs_cube = np.array(data_filtered['energy1'])
            e2_for_pairs_cube = np.array(data_filtered['energy2'])
            sep_for_pairs_cube = np.array(data_filtered['separation'])
            detections_for_pairs_cube = np.array(data_filtered['ndetected'])
            # NUMPY array as (sep, e2, e1) so that FITS is (e1,e2,sep)
            coords_for_pairs_cube = np.vstack((sep_for_pairs_cube, e2_for_pairs_cube, e1_for_pairs_cube)).T  # sta
            pairs_cube_detections_iw_io, pairs_cube_edges = np.histogramdd(coords_for_pairs_cube,
```

```
                                                    bins=[seps_bin_edges, energy_bin_edges, energy_bin_edges],
                                                    weights=detections_for_pairs_cube)  # create a 3D histogra
            # BE CAREFUL: if one of the separation bins is empty, it must be because:
            # 1. there are no detections for this separation (true 0)
            # 2. there are no simulated pairs for this separation (false 0) -> replace 0s with NaN
            for i, sep_edge in enumerate(seps_bin_edges[:-1]):  # iterate through bin edges except the last one
                # check if any sep_for_pairs_cube is inside the bin
                next_sep_edge = seps_bin_edges[i + 1]
                if not np.any((sep_for_pairs_cube >= sep_edge) & (sep_for_pairs_cube < next_sep_edge)):
                    # if not, replace the corresponding slice in pairs_cube_detections_iw_io with NaN
                    pairs_cube_detections_iw_io[i, :, :] = np.nan  # replace the slice with NaN
                    aux.vprint(f"   In cube creation: Replacing slice for separation bin {sep_edge}-{next_sep_edge

            # create a FITS file for the pairs cube detections
            hdu = fits.PrimaryHDU(pairs_cube_detections_iw_io)
            hdu.header['XIFU_CFG'] = xifu_config
            hdu.header['FLUX'] = sim_source_flux
            hdu.header['MAXPIXEL'] = main_pixel_num
            hdu.writeto(pairs_cube_detections_iw_io_file, overwrite=True)
        else:
            aux.vprint(f"Pairs cube detections file {pairs_cube_detections_iw_io_file} already exists. Will read i
            # read the pairs cube detections from the FITS file
            with fits.open(pairs_cube_detections_iw_io_file) as hdul:
                pairs_cube_detections_iw_io = hdul[0].data

        # correct the PAIRS cube for the simulated source
        if use_sim_source:
            # multiply this cube by the probability cube element wise
            cube_detections_iw_io = pairs_cube_detections_iw_io * prob_cube
            corrected_cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/corrected_cubes/corrected_cube_
            if not os.path.exists(corrected_cube_detections_iw_io_file):
                aux.vprint(f"Corrected pairs cube detections file {corrected_cube_detections_iw_io_file} does not
                # save the corrected cube to
                hdu = fits.PrimaryHDU(cube_detections_iw_io)
                hdu.header['XIFU_CFG'] = xifu_config
                hdu.header['FLUX'] = sim_source_flux
                hdu.header['MAXPIXEL'] = main_pixel_num
                hdu.writeto(corrected_cube_detections_iw_io_file, overwrite=True)
        else:
            #vprint(f"Using pairs cube detections file {pairs_cube_detections_iw_io_file} as is (no correction for
            cube_detections_iw_io = pairs_cube_detections_iw_io

        # Create a collapsed cube for the pairs cube detections
        if win in windows_for_collapsed_cube:
            for isep, sep in enumerate(use_separations):
                detections_slice_sep = cube_detections_iw_io[isep, :, :]  # slice for the current separation
                prob_detections_slice_sep = detections_slice_sep / nsimulated_1e1_1e2_1sep
                if use_sim_source:
                    cube_e1e2collapsed[isep, io, iw] = np.sum(prob_detections_slice_sep)  # store the mean of the
                else:
                    cube_e1e2collapsed[isep, io, iw] = np.nanmean(prob_detections_slice_sep)
                #print(f"    Window: {win}, Offset: {off}, Separation: {sep}, cube_e1e2collapsed[{isep}, {io_plot}
```

Calculate the slice for win=0 and off=0 if plotting

In [ ]:
```
if sep_for_plot_mosaic > 0 and use_sim_source:
    # read CUBE FITS file for the separation and win=0, offset=0
    if use_sim_source:
        cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/corrected_cubes/corrected_cube_detections_win
    else:
        cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/pairs_cubes/pairs_cube_detections_win0_off0.f
    with fits.open(cube_detections_iw_io_file) as hdul:
        cube_detections_iw_io = hdul[0].data
    # create a slice for the mosaic plot
    separation_index = np.where(np.array(use_separations) == sep_for_plot_mosaic)[0][0]  # get the index of the se
    slice_to_plot = cube_detections_iw_io[separation_index, :, :]
    prob_slice_to_plot_win0_off0 = slice_to_plot / nsimulated_1e1_1e2_1sep  # use the corrected slice if available
```

Plot the mosaic of differential-detection images for win/off combinations

In [ ]:
```
if sep_for_plot_mosaic > 0:

    # do not consider window=0 for the mosaic plot if use_sim_source is True
    if use_sim_source:
        # remove window=0 from the list of windows for the mosaic plot
```

```
        windows_for_mosaic = [w for w in windows if w != 0]
    else:
        windows_for_mosaic = windows
    ncols_mosaic = len(windows_for_mosaic)  # update the number of columns for the mosaic plot based on the window

    # create a mosaic figure (with squared plots) of the same slice in different data-cubes for each window and of
    fig_mosaic, ax_mosaic = plt.subplots(nrows_mosaic, ncols_mosaic, figsize=(18, 12), sharex=True, sharey=True)
    #fig_mosaic.suptitle(f'Mosaic of probability of detection (config: {xifu_config=}, {th=}, {sUp=}, {sDown=}, {s
    fig_mosaic.suptitle(f'Differential probability of detection (wrt win=0, off=0)', fontsize=10)
    e12_labs = []
    for ilab in range(len(energy_bin_centers)):
        if energy_bin_centers[ilab] < 1:
            e12_labs.append(f'{energy_bin_centers[ilab]:.1f}')
        else:
            e12_labs.append(f'{energy_bin_centers[ilab]:.0f}')

    # vprint("Window    Offset    DataCube       CorrDataCube       DataSlice")

    # log scale normalization
    cmap = plt.get_cmap('viridis')
    if use_sim_source:
        vmin = 1E-5 # minimum value for normalization (to avoid log(0))
        vmax = 1
        #norm = mcolors.LogNorm(vmin=vmin, vmax=vmax)
        norm = SymLogNorm(linthresh=5e-7, linscale=1, vmin=-1E-5, vmax=0.2, base=10)
        neg_cmap = plt.get_cmap('Greys_r', 128)   # reversed grey for negatives
        pos_cmap = plt.get_cmap('viridis', 128)
        colors = np.vstack((
            neg_cmap(np.linspace(0.4, 1, 128)),  # grey for neg
            pos_cmap(np.linspace(0, 1, 128))     # green-yellow for pos
        ))
        custom_cmap = LinearSegmentedColormap.from_list('asym_div_viridis', colors)
    else:
        vmin = 1E-2
        vmax = 1
        norm = mcolors.Normalize(vmin=0, vmax=vmax)
        #norm = mcolors.LogNorm(vmin=vmin, vmax=vmax)
        custom_cmap = cmap  # use the default colormap

    # Plot each window and offset
    # ==========================================

    for io in range(len(offsets)):
        for iw in range(len(windows_for_mosaic)):
            # get offset in inverse order for plotting reasons: mosaic plots would otherwise show the first offset
            io_plot = len(offsets) - 1 - io
            off = offsets[io_plot]  # use the current offset for plotting
            win = windows_for_mosaic[iw]  # use the current window for plotting (skip window=0)
            print(f"Offset: {off}, Window: {win}")

            if win == 0 and off > 0:
                # skip this combination, as it is not meaningful
                aux.vprint(f"    Skipping window {win} with offset {off} as it is not applicable.")
                continue

            if win in old_windows and off > 5:
                print(f"    Skipping window {win} with offset {off} as it is not applicable.")
                if sep_for_plot_mosaic > 0:
                    ax_mosaic[io,iw].xaxis.set_visible(False)
                    ax_mosaic[io,iw].yaxis.set_visible(False)
                continue

            # read the pairs cube detections for the current window and offset
            if use_sim_source:
                cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/corrected_cubes/corrected_cube_detect
            else:
                cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/pairs_cubes/pairs_cube_detections_win
            # read cube_detections_iw_io from the FITS file
            with fits.open(cube_detections_iw_io_file) as hdul:
                cube_detections_iw_io = hdul[0].data

            # plot mosaic of slices of the data cube
            # ---------------------------------------
            separation_index = np.where(np.array(use_separations) == sep_for_plot_mosaic)[0][0]
            slice_to_plot = cube_detections_iw_io[separation_index,:, :]
            prob_slice_to_plot = slice_to_plot / nsimulated_1e1_1e2_1sep  # use the corrected slice if available
            data = prob_slice_to_plot
```

```
            if use_sim_source:
                diff_prob_slice_to_plot = prob_slice_to_plot - prob_slice_to_plot_win0_off0
                data = diff_prob_slice_to_plot  # use the difference from the first bin for plotting
                # if there is a NaN in the slice, print a warning
                if np.isnan(diff_prob_slice_to_plot).any():
                    print(f"Warning: NaN found in the slice for window {win}, offset {off}, separation {sep_for_pl

            #print(f"       Data range: {np.min(data)}, {np.max(data)}")

            # create the image with the specified colormap and normalization using pcolormesh (bins of different s
            if aux.is_notebook:
                im = ax_mosaic[io, iw].imshow(data, aspect='auto', origin='lower', cmap=custom_cmap, norm=norm, in
                ax_mosaic[io, iw].set_xticks(np.arange(len(energy_bin_centers)))
                ax_mosaic[io, iw].set_xticklabels(e12_labs, rotation=45, fontsize=6)
                ax_mosaic[io, iw].set_yticks(np.arange(len(energy_bin_centers)))
                ax_mosaic[io, iw].set_yticklabels(e12_labs, fontsize=6)
            else:
                X, Y = np.meshgrid(energy_bin_edges, energy_bin_edges)
                im = ax_mosaic[io, iw].pcolormesh(X, Y, data, cmap=custom_cmap, norm=norm, shading='auto')
                ax_mosaic[io, iw].set_xticks(energy_bin_centers[2:])  # set x-ticks only after the first two bins
                ax_mosaic[io, iw].set_yticks(energy_bin_centers[2:])  # set y-ticks only after the first two bins
                ax_mosaic[io, iw].set_xticklabels(e12_labs[2:], rotation=45, fontsize=6)
                ax_mosaic[io, iw].set_yticklabels(e12_labs[2:], fontsize=6)
            ax_mosaic[io, iw].set_title(f'Window: {win}, Offset: {off}', fontsize=7)
            ax_mosaic[io, iw].set_xlabel('Energy primary (keV)', fontsize=7)
            ax_mosaic[io, iw].set_ylabel('Energy secondary (keV)', fontsize=7)
            ax_mosaic[io, iw].set_aspect('equal')

            # add color bar to each subplot
            cbar = fig_mosaic.colorbar(plt.cm.ScalarMappable(norm=norm, cmap=custom_cmap), ax=ax_mosaic[io, iw], f
            cbar.ax.tick_params(labelsize=5)  # adjust color bar tick label size
    # adjust layout
    plt.tight_layout()
    plt.show()
```

In [ ]:

```
"""
from matplotlib.ticker import LogFormatterSciNotation
is_notebook = True
formatter = LogFormatterSciNotation()

# initialize plotting area
if sep_for_plot_mosaic > 0:
    sep = sep_for_plot_mosaic  # select the first separation for the mosaic
    # create a mosaic figure (with squared plots) of the same slice in different data-cubes for each window and of
    fig_mosaic, ax_mosaic = plt.subplots(nrows, ncols, figsize=(18, 12), sharex=True, sharey=True)
    #fig_mosaic.suptitle(f'Mosaic of probability of detection (config: {xifu_config=}, {th=}, {sUp=}, {sDown=}, {s
    fig_mosaic.suptitle(f'Differential probability of detection (wrt win=0, off=0)', fontsize=10)
    e12_labs = []
    for ilab in range(len(energy_bin_centers)):
        if energy_bin_centers[ilab] < 1:
            e12_labs.append(f'{energy_bin_centers[ilab]:.1f}')
        else:
            e12_labs.append(f'{energy_bin_centers[ilab]:.0f}')

    # calculate the slice for win=0 and offset=0
    # ==========================================
    print(f"Calculating slice for win=0 and offset=0 for separation {sep_for_plot_mosaic} samples"
          )
    pickle_file = f'{pickles_dir}/detectedFakes_win0_off0.pkl'
    # read detection data from the pickle file
    with open(pickle_file, 'rb') as f:
        data = pickle.load(f)
        #print(data)

    use_separations = relevant_separations  # use the relevant separations for the analysis
    # for window=0 and offset=0, we have a different data structure
    data_table = table.Table(rows=data, names=('separation', 'energy1', 'energy2', 'samplesDown', 'samplesUp', 'th
    data_filtered = data_table[(data_table['threshold'] == th) & (data_table['samplesUp'] == sUp) & (data_table['s

    # get only separations that are in the relevant separations list
    data_filtered = data_filtered[np.isin(data_filtered['separation'], use_separations)]
    #print(f"Filtered data: {data_filtered}")
    # create a data cube for each pickle file (and optionally save it to a FITS file)
    #  (use histogramdd to create a cube of detections)
    e1_for_pairs_cube = np.array(data_filtered['energy1'])
    e2_for_pairs_cube = np.array(data_filtered['energy2'])
    sep_for_pairs_cube = np.array(data_filtered['separation'])
```

```
    detections_for_pairs_cube = np.array(data_filtered['ndetected'])
    # NUMPY array as (sep, e2, e1) so that FITS is (e1,e2,sep)
    coords_for_pairs_cube = np.vstack((sep_for_pairs_cube, e2_for_pairs_cube, e1_for_pairs_cube)).T  # stack the c
    pairs_cube_detections_iw_io, pairs_cube_edges = np.histogramdd(coords_for_pairs_cube,
                                                   bins=[seps_bin_edges, energy_bin_edges, energy_bin_edges],
                                                   weights=detections_for_pairs_cube)  # create a 3D histogram with t
    # BE CAREFUL: if one of the separation bins is empty, it must be because:
    # 1. there are no detections for this separation (true 0)
    # 2. there are no simulated pairs for this separation (false 0) -> replace 0s with NaN

    for i, sep_edge in enumerate(seps_bin_edges[:-1]):  # iterate through bin edges except the last one
        # check if any sep_for_pairs_cube is inside the bin
        next_sep_edge = seps_bin_edges[i + 1]
        if not np.any((sep_for_pairs_cube >= sep_edge) & (sep_for_pairs_cube < next_sep_edge)):
            # if not, replace the corresponding slice in pairs_cube_detections_iw_io with NaN
            pairs_cube_detections_iw_io[i, :, :] = np.nan  # replace the slice with NaN
            print(f"   In cube creation: Replacing slice for separation bin {sep_edge}-{next_sep_edge} (not presen
    # correct the PAIRS cube for the simulated source
    if use_sim_source:
        # multiply this cube by the probability cube element wise
        cube_detections_iw_io = pairs_cube_detections_iw_io * prob_cube
    else:
        # use the data cube as is
        cube_detections_iw_io = pairs_cube_detections_iw_io

    # create a slice for the mosaic plot
    separation_index = np.where(np.array(use_separations) == sep_for_plot_mosaic)[0][0]  # get the index of the se
    slice_to_plot = cube_detections_iw_io[separation_index, :, :]
    prob_slice_to_plot_win0_off0 = slice_to_plot / nsimulated_1e1_1e2_1sep  # use the corrected slice if available


# Run the analysis for each window and offset
# ==========================================
# vprint("Window    Offset    DataCube       CorrDataCube       DataSlice")
for io in range(len(offsets)):
    #for iw in range(len(windows)-1):
    for iw in range(ncols):  # use ncols if use_sim_source is True
        # get offset in inverse order for plotting reasons: mosaic plots would otherwise show the first offset at
        io_plot = len(offsets) - 1 - io
        off = offsets[io_plot]  # use the current offset for plotting
        if use_sim_source:
            iw_plot = iw + 1  # use the current window for plotting (skip window=0)
        else:
            iw_plot = iw
        win = windows[iw_plot]  # use the current window for plotting (skip window=0)
        print(f"Offset: {off}, Window: {win}")

        # check if the window and offset are applicable
        #if win == 0 and off > 0:
        #if win == 0:
        #    print(f"    Skipping window {win} with offset {off} as it is not applicable.")
            #if sep_for_plot_mosaic > 0:
            #    ax_mosaic[io,iw].xaxis.set_visible(False)
            #    ax_mosaic[io,iw].yaxis.set_visible(False)
        #    continue
        if win in old_windows and off > 5:
            print(f"    Skipping window {win} with offset {off} as it is not applicable.")
            if sep_for_plot_mosaic > 0:
                ax_mosaic[io,iw].xaxis.set_visible(False)
                ax_mosaic[io,iw].yaxis.set_visible(False)
            continue
        pickle_file = f'{pickles_dir}/detectedFakes_win{win}_off{off}.pkl'

        # read detection data from the pickle file
        with open(pickle_file, 'rb') as f:
            data = pickle.load(f)
            #print(data)

            use_separations = relevant_separations  # use the relevant separations for the analysis
            if win == 0 and off == 0:
                # for window=0 and offset=0, we have a different data structure
                data_table = table.Table(rows=data, names=('separation', 'energy1', 'energy2', 'samplesDown', 'sam
                data_filtered = data_table[(data_table['threshold'] == th) & (data_table['samplesUp'] == sUp) & (d
            else:
                data_table = table.Table(rows=data, names=('separation', 'energy1', 'energy2', 'window', 'offset',
                data_filtered = data_table.copy()
```

```
        # get only separations that are in the relevant separations list
        data_filtered = data_filtered[np.isin(data_filtered['separation'], use_separations)]

        #print(f"Filtered data: {data_filtered}")

        #look for rows where nfake > 0
        data_filtered_nfake = data_filtered[data_filtered['nfake'] > 0]
        if len(data_filtered_nfake) > 0:
            print(f"Filtered data with nfake > 0: {data_filtered_nfake}")

        # create a data cube for each pickle file (and optionally save it to a FITS file)
        #  (use histogramdd to create a cube of detections)
        e1_for_pairs_cube = np.array(data_filtered['energy1'])
        e2_for_pairs_cube = np.array(data_filtered['energy2'])
        sep_for_pairs_cube = np.array(data_filtered['separation'])
        detections_for_pairs_cube = np.array(data_filtered['ndetected'])
        # NUMPY array as (sep, e2, e1) so that FITS is (e1,e2,sep)
        coords_for_pairs_cube = np.vstack((sep_for_pairs_cube, e2_for_pairs_cube, e1_for_pairs_cube)).T  # stack t
        pairs_cube_detections_iw_io, pairs_cube_edges = np.histogramdd(coords_for_pairs_cube,
                                                    bins=[seps_bin_edges, energy_bin_edges, energy_bin_edges],
                                                    weights=detections_for_pairs_cube)  # create a 3D histogram wi
        # BE CAREFUL: if one of the separation bins is empty, it must be because:
        # 1. there are no detections for this separation (true 0)
        # 2. there are no simulated pairs for this separation (false 0) -> replace 0s with NaN
        for i, sep_edge in enumerate(seps_bin_edges[:-1]):  # iterate through bin edges except the last one
            # check if any sep_for_pairs_cube is inside the bin
            next_sep_edge = seps_bin_edges[i + 1]
            if not np.any((sep_for_pairs_cube >= sep_edge) & (sep_for_pairs_cube < next_sep_edge)):
                # if not, replace the corresponding slice in pairs_cube_detections_iw_io with NaN
                pairs_cube_detections_iw_io[i, :, :] = np.nan  # replace the slice with NaN
                print(f"   In cube creation: Replacing slice for separation bin {sep_edge}-{next_sep_edge} (not pr

        if create_cubes:
            pairs_cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/pairs_cube_detections_win{win}_off{
            hdu = fits.PrimaryHDU(pairs_cube_detections_iw_io)
            hdu.header['XIFU_CFG'] = xifu_config
            hdu.header['FLUX'] = sim_source_flux
            hdu.header['MAXPIXEL'] = main_pixel_num
            hdu.writeto(pairs_cube_detections_iw_io_file, overwrite=True)

        # correct the PAIRS cube for the simulated source
        if use_sim_source:
            # multiply this cube by the probability cube element wise
            cube_detections_iw_io = pairs_cube_detections_iw_io * prob_cube
            # if nan in cube_detections_iw_io, print where there is a nan and raise an error

            if create_cubes:
                corrected_cube_detections_iw_io_file = f"{analysis_dir}/analysis_pairs/corrected_cube_detections_w
                hdu = fits.PrimaryHDU(cube_detections_iw_io)
                hdu.header['XIFU_CFG'] = xifu_config
                hdu.header['FLUX'] = sim_source_flux
                hdu.header['MAXPIXEL'] = main_pixel_num
                hdu.writeto(corrected_cube_detections_iw_io_file, overwrite=True)
        else:
            # use the data cube as is
            cube_detections_iw_io = pairs_cube_detections_iw_io

        if win in windows_for_collapsed_cube:
            for isep, sep in enumerate(use_separations):
                detections_slice_sep = cube_detections_iw_io[isep, :, :]  # slice for the current separation
                prob_detections_slice_sep = detections_slice_sep / nsimulated_1e1_1e2_1sep
                if use_sim_source:
                    cube_e1e2collapsed[isep, io_plot, iw] = np.sum(prob_detections_slice_sep)  # store the mean of
                else:
                    cube_e1e2collapsed[isep, io_plot, iw] = np.nanmean(prob_detections_slice_sep)
                #print(f"    Window: {win}, Offset: {off}, Separation: {sep}, cube_e1e2collapsed[{isep}, {io_plot}


        if sep_for_plot_mosaic > 0:
            im_list = []
            # plot mosaic of slices of the data cube
            # ---------------------------------------

            # create a new figure for each window and offset using the slice for the separation_index
            separation_index = np.where(np.array(use_separations) == sep_for_plot_mosaic)[0][0]
            slice_to_plot = cube_detections_iw_io[separation_index,:, :]
            prob_slice_to_plot = slice_to_plot / nsimulated_1e1_1e2_1sep  # use the corrected slice if available
```

```
            diff_prob_slice_to_plot = prob_slice_to_plot - prob_slice_to_plot_win0_off0
            # if there is a NaN in the slice, print a warning
            if np.isnan(diff_prob_slice_to_plot).any():
                print(f"Warning: NaN found in the slice for window {win}, offset {off}, separation {sep_for_plot_m
                #print(f"    Slice shape: {slice_to_plot.shape}, prob_slice_to_plot shape: {prob_slice_to_plot.sha

            # print an example value of all cubes
            #print(f"Window: {win}, Offset: {off} => ")
            #print(f"              Ex. DataCube: {data_cube_detections_iw_io[separation_index, 0, 0]}")
            #print(f"              Ex. Corrected DataCube: {corrected_data_cube_detections_iw_io[separation_index,
            #print(f"              Ex. DataSlice: {data_slice_sep[0, 0]}")
            #print(win, off, data_cube_detections_iw_io[separation_index, 0, 0], corrected_data_cube_detections_iw

            #print(f"    Plotting mosaic for window {win}, offset {off}, separation {sep_for_plot_mosaic} (index {

            # log scale normalization
            cmap = plt.get_cmap('viridis')


            if use_sim_source:
                vmin = 1E-5 # minimum value for normalization (to avoid log(0))
                vmax = 1
                #norm = mcolors.LogNorm(vmin=vmin, vmax=vmax)
                norm = SymLogNorm(linthresh=5e-7, linscale=1, vmin=-1E-5, vmax=0.2, base=10)
                neg_cmap = plt.get_cmap('Greys_r', 128)   # reversed grey for negatives
                pos_cmap = plt.get_cmap('viridis', 128)
                colors = np.vstack((
                    neg_cmap(np.linspace(0.4, 1, 128)),  # grey for neg
                    pos_cmap(np.linspace(0, 1, 128))     # green-yellow for pos
                ))
                custom_cmap = LinearSegmentedColormap.from_list('asym_div_viridis', colors)
            else:
                vmin = 0.5
                vmax = 1
                norm = mcolors.Normalize(vmin=vmin, vmax=vmax)
                custom_cmap = cmap  # use the default colormap

            # use meshgrid to create the X and Y coordinates for the image
            X, Y = np.meshgrid(energy_bin_edges, energy_bin_edges)
            if win == 0 and off == 0:
                # for window=0 and offset=0
                data = prob_slice_to_plot
            else:
                # use the difference from the first bin for plotting
                #masked_data = np.ma.masked_less_equal(diff_prob_slice_to_plot, 0)
                #data = masked_data
                data = diff_prob_slice_to_plot
                # if negative values, print a warning
                #if np.any(data < 0):
                #    print(f"Warning: Min data for window {win}, offset {off} is negative: {np.min(data)}")
                #print(f"      Example value for diff plotting: {data[0, 0]}")
            #print(f"       Data range: {np.min(data)}, {np.max(data)}")

            # create the image with the specified colormap and normalization using pcolormesh (bins of different s
            if is_notebook:
                im = ax_mosaic[io, iw].imshow(data, aspect='auto', origin='lower', cmap=custom_cmap, norm=norm, in
                im_list.append(im)  # append the image to the list for colorbar later
                ax_mosaic[io, iw].set_xticks(np.arange(len(energy_bin_centers)))
                ax_mosaic[io, iw].set_xticklabels(e12_labs, rotation=45, fontsize=6)
                ax_mosaic[io, iw].set_yticks(np.arange(len(energy_bin_centers)))
                ax_mosaic[io, iw].set_yticklabels(e12_labs, fontsize=6)
            else:
                im = ax_mosaic[io, iw].pcolormesh(X, Y, data, cmap=custom_cmap, norm=norm, shading='auto')
                ax_mosaic[io, iw].set_xticks(energy_bin_centers[2:])  # set x-ticks only after the first two bins
                ax_mosaic[io, iw].set_yticks(energy_bin_centers[2:])  # set y-ticks only after the first two bins
                ax_mosaic[io, iw].set_xticklabels(e12_labs[2:], rotation=45, fontsize=6)
                ax_mosaic[io, iw].set_yticklabels(e12_labs[2:], fontsize=6)
            ax_mosaic[io, iw].set_title(f'Window: {win}, Offset: {off}', fontsize=7)
            ax_mosaic[io, iw].set_xlabel('Energy primary (keV)', fontsize=7)
            ax_mosaic[io, iw].set_ylabel('Energy secondary (keV)', fontsize=7)
            ax_mosaic[io, iw].set_aspect('equal')

            # add color bar to each subplot
            cbar = fig_mosaic.colorbar(plt.cm.ScalarMappable(norm=norm, cmap=custom_cmap), ax=ax_mosaic[io, iw], f
            cbar.ax.tick_params(labelsize=5)  # adjust color bar tick label size

if sep_for_plot_mosaic > 0:
```

```
    # adjust layout
    plt.tight_layout()
    plt.show()
"""
```

In [ ]:
```
if sep_for_plot_mosaic > 0:
    # save the mosaic figure (png and PDF)
    if use_sim_source:
        fig_mosaic.savefig(f'analysis_pairs/Figures/mosaic_prob_detected_events_slices_sep{sep_for_plot_mosaic}_wi
        fig_mosaic.savefig(f'analysis_pairs/Figures/mosaic_prob_detected_events_slices_sep{sep_for_plot_mosaic}_wi
    else:
        fig_mosaic.savefig(f'analysis_pairs/Figures/mosaic_prob_detected_events_slices_sep{sep_for_plot_mosaic}_wi
        fig_mosaic.savefig(f'analysis_pairs/Figures/mosaic_prob_detected_events_slices_sep{sep_for_plot_mosaic}_wi
```

Collapse fraction_detected_pulses cube

1. Take mean value along separations axis
2. Plot collapsed image

In [ ]:
```
# print the shape of the corrected cube -> should be (separations, offsets, windows)
print(f"cube_e1e2collapsed: {cube_e1e2collapsed.shape}")
# collase cube in axis 0 to get the mean of detected fractions along the separations and plot image (account for n
cube_collapsed = np.nanmean(cube_e1e2collapsed, axis=0)  # collapse the cube along the separations axis
print(f"Collapsed cube shape: {cube_collapsed.shape}")
vmin = np.nanmin(cube_collapsed)
vmax = np.nanmax(cube_collapsed)*1.01
cmap_collapsed = plt.get_cmap('viridis').copy()  # colormap for the collapsed cube
cmap_collapsed.set_bad(color='white')  # set NaN values to white in the colormap
print(f"Collapsed cube range: {vmin}, {vmax}")

# create a new figure for the collapsed cube
fig_collapsed, ax_collapsed = plt.subplots(figsize=(8, 6))
# create a normalization for the color map
norm_collapsed = mcolors.Normalize()
if use_sim_source:
    # use a two-slope normalization for the collapsed cube
    norm_collapsed = mcolors.TwoSlopeNorm(vmin=vmin, vcenter=0.88, vmax=vmax)
else:
    # use a linear normalization for the collapsed cube
    norm_collapsed = mcolors.Normalize(vmin=vmin, vmax=vmax)

# mask invalid values in the collapsed cube to avoid errors while hoovering with the mouse
masked_cube_collapsed = np.ma.masked_invalid(cube_collapsed)
# plot the collapsed cube
im_collapsed = ax_collapsed.imshow(masked_cube_collapsed, origin='lower', aspect='auto', cmap=cmap_collapsed, norm
#ax_collapsed.set_title(f'Collapsed Fraction of detected (config: {xifu_config=}, {th=}, {sUp=}, {sDown=})', fonts
ax_collapsed.set_title(f'Collapsed Fraction of detected pulses', fontsize=10)
ax_collapsed.set_ylabel('Offset (samples)')
ax_collapsed.set_xlabel('Window (samples)')
ax_collapsed.set_yticks(np.arange(len(offsets)))
ax_collapsed.set_yticklabels(offsets, rotation=45, fontsize=8)
ax_collapsed.set_xticks(np.arange(len(windows_for_collapsed_cube)))
ax_collapsed.set_xticklabels(windows_for_collapsed_cube, fontsize=8)
# add color bar to the collapsed plot
cbar_collapsed = fig_collapsed.colorbar(plt.cm.ScalarMappable(norm=norm_collapsed, cmap=cmap_collapsed), ax=ax_col
cbar_collapsed.ax.tick_params(labelsize=8)
plt.tight_layout()
plt.show()
```

In [ ]:
```
if use_sim_source:
    # save the collapsed figure (png and PDF) with the simulated source flux
    fig_collapsed.savefig(f'analysis_pairs/Figures/collapsed_fraction_detected_pulses_cube_windows_offsets_{sim_so
    fig_collapsed.savefig(f'analysis_pairs/Figures/collapsed_fraction_detected_pulses_cube_windows_offsets_{sim_so
else:
    # save the collapsed figure (png and PDF)
    fig_collapsed.savefig(f'analysis_pairs/Figures/collapsed_fraction_detected_pulses_cube_windows_offsets.png', d
    fig_collapsed.savefig(f'analysis_pairs/Figures/collapsed_fraction_detected_pulses_cube_windows_offsets.pdf', d
```

Some DUMB tests

In [ ]:
```
if aux.is_notebook:
    win= 6
    off = 0
```

```
    # print values of cube_collapsed for window=win, offset=off and separation=sep_for_plot_mosaic
    isep = np.where(np.array(relevant_separations) == sep_for_plot_mosaic)[0][0]  # find the index of separation=s
    iw = windows_for_collapsed_cube.index(win)  # find the index of window=win
    io = np.where(np.array(offsets) == off)[0][0]  # find the index of offset=off

    print(f"cube_collapsed[io, iw]: {cube_collapsed[io, iw]:.2f}")
    print(f"cube_e1e2collapsed[isep, io, iw]: {cube_e1e2collapsed[isep, io, iw]:.2f}")
```

In [ ]:

```
if aux.is_notebook:
    ########### TESTING PART ###########
    # This part is for testing purposes only, to read the data from a pickle file and print
    # the filtered data and an example row with specific values.
    # It should not be part of the main code execution.

    #read the data for window=20 and offset=0
    win = 0
    off = 0
    e1 = 0.2
    e2 = 0.2
    pickle_file = f'{pickles_dir}/detectedFakes_win{win}_off{off}.pkl'
    # read the data from the pickle file
    with open(pickle_file, 'rb') as f:
        data = pickle.load(f)
        if win == 0 and off == 0:
            # for window=0 and offset=0, we have a different data structure
            data_table = table.Table(rows=data, names=('separation', 'energy1', 'energy2', 'samplesDown', 'samples
            data_filtered = data_table[(data_table['threshold'] == th) & (data_table['samplesUp'] == sUp) & (data_
        else:
            # for other windows and offsets, we have a different data structure
            data_table = table.Table(rows=data, names=('separation', 'energy1', 'energy2', 'window', 'offset', 'nd
            data_filtered = data_table.copy()
    #print(f"Filtered data: {data_filtered}")
    # print row with seaparation=20, energy1=0.2, energy2=0.5
    example_row = data_filtered[(data_filtered['separation'] == sep_for_plot_mosaic) & (data_filtered['energy1'] =
    print(f"Example row:\n {example_row}")
```

In [ ]: